

Corso di Programmazione Web

Esame - 6 Giugno 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.
- **Scrivere nome, cognome, numero di matricola nel foglio dove si svolge l'esame**

Sistema per la gestione di prestiti di oggetti tra utenti

Ci è stato richiesto di progettare un'applicazione web che consente agli utenti di **prestare e ricevere in prestito oggetti** (libri, strumenti, videogiochi, ecc.) in modo tracciabile.

Requisiti dell'applicazione

Il sistema deve prevedere:

- Un'API per registrare un nuovo oggetto da prestare (id, nome, descrizione, categoria, disponibilità, id proprietario);
- Un'API per visualizzare la lista degli oggetti disponibili;
- Un'API per richiedere il prestito di un oggetto (id oggetto, id richiedente, data inizio, durata prevista);
- Un'API per vedere la cronologia dei prestiti effettuati o ricevuti su un oggetto.

Si progetti in modo che gli id siano assegnati dal server nel momento in cui si registra un oggetto. Gli id degli utenti (proprietari e richiedenti di prestiti) sono numerici. Il campo disponibilità serve a indicare se l'oggetto è disponibile o no (non la quantità).

Per definire le API, si ipotizzi che il prestito sia automaticamente accettato senza fare verifiche sulla disponibilità.

Domanda 1: Definizione delle API (15 punti)

Per ciascuna API:

- **3 punti totali:** Modelli di dati richieste/risposte con tipi e vincoli (es, massimo, minimo). Spiegare il motivo di eventuali vincoli.
- **2 punti per API:** Descrizione endpoint (URL + metodo HTTP)
- **1 punto per API:** Esempio concreto di richiesta con successo e risposta del server. Si indichi il codice di risposta, ed eventuali JSON di risposta.

Domanda 2: Gestione degli Errori (5 punti)

- **1 punto totale:** Spiegare i codici di errore più rilevanti (400, 404, 500)
- **2 punti per esempio, max 4 punti:** Fornire **due esempi** di richieste che possono causare uno dei precedenti errori nelle API appena progettate.

Domanda 3: Concorrenza (3 punti)

- **3 punti totali:** Il sistema deve impedire che uno stesso oggetto venga prestato contemporaneamente a più utenti. *Come si potrebbe progettare una logica robusta per la gestione della concorrenza su oggetti condivisi?*

Modello Oggetto

```
{
  "objectId": int,
  "name": string,
  "description": string,
  "category": string,
  "available": boolean,
  "ownerId": int
}
```

Vincoli e descrizione:

- objectId: assegnato dal server in modo incrementale
- name: obbligatorio, max 20 caratteri
- category: opzionale, valori predefiniti (es. "libro", "strumento", "gioco")
- available: default true, viene modificato quando l'oggetto viene dato in prestito; va bene anche una stringa, ma con vincolo di due soli valori possibili; oppure un intero con vincolo >0, per mostrare quanti oggetti sono disponibili.
- ownerId: specifica l'utente proprietario

Modello Richiesta Prestito

```
{
  "requestId": int,
  "objectId": int,
  "borrowerId": int,
  "startDate": datetime,
  "durationDays": int, min=1, max=30
}
```

Vincoli e descrizione:

- requestId, identifica la richiesta, viene assegnato dal server
- objectId: deve corrispondere a un oggetto registrato
- borrowerId: intero, specifica l'utente che chiede in prestito l'oggetto
- startDate: datetime, specifica la partenza del prestito
- durationDays: intero, massimo 30 giorni e minimo 1 giorno, specifica la durata del prestito; va bene anche un datetime, purché si specifichi il vincolo di durata massima e che data fine > data inizio.

API 1: Registrazione nuovo oggetto

- Metodo: POST /objects
- Richiesta:

```
{
  "name": "Chitarra classica",
```

```
"description": "Buone condizioni",
"category": "strumento",
"ownerId": 2
}
```

L'objectId e il campo available sono assegnati dal server nel momento in cui viene registrato l'oggetto. Opzionalmente, il campo available si può far inserire all'utente.

Risposta: 201, Created

```
{
  "objectId": 1,
  "message": "Oggetto registrato con successo"
}
```

API 2: Lista oggetti disponibili

- Metodo: GET /objects
- Risposta:

```
[
  {
    "objectId": 1,
    "name": "Chitarra classica",
    "description": "Buone condizioni",
    "category": "strumento",
    "ownerId": 2
    "available": true
  },
  ...
]
```

API 3: Richiesta di prestito

- Metodo: POST /loans
- Richiesta:

```
{
  "objectId": 1,
  "borrowerId": 4,
  "startDate": "2025-06-10",
  "durationDays": 7
}
```

Il campo durationDays contiene un vincolo per verificare che la durata sia tra 1 e 30 giorni.

Risposta:

```
{
  "requestId": 15,
  "message": "Richiesta di prestito approvata."
}
```

```
}
```

Questa API cambia lo stato dell'oggetto identificato a "available": False fino alla fine del prestito.

API 4: Cronologia prestiti per oggetto

- **Metodo:** GET /objects/1/loans

Risposta:

```
[
  {
    "requestId": 15,
    "borrowerId": 4,
    "status": "accepted",
    "startDate": "2025-06-10",
    "durationDays": 7
  },
  ...
]
```

Gestione degli errori

Codici errore rilevanti:

- 400 Bad Request / 422 Unprocessable: dati mancanti o malformati, per esempio una richiesta POST senza body o dei campi che non rispettano le specifiche.
- 404 Not Found: risorsa non trovata, l'utente ha chiesto una risorsa che non esiste
- 500 Internal Server Error: errore generico del server, per esempio un malfunzionamento o mancata connessione

Esempio errore 1 – richiesta malformata

Richiesta:

```
{
  "objectId": 1,
  "borrowerId": "quattro",
  "startDate": "10/06/2025",
  "durationDays": 40
}
```

Risposta:

```
{
  "error": "Dati non validi: ID utente deve essere un intero, durata massima 30 giorni"
}
```

Esempio errore 2 – oggetto non disponibile

Richiesta:

```
{
```

```
"objectId": 100,  
"borrowerId": 4,  
"startDate": "2025-06-10",  
"durationDays": 5  
}
```

Risposta:

```
{  
  "error": "Oggetto 100 inesistente."  
}
```

Domanda 3: Concorrenza (3 punti)

La concorrenza può essere gestita tramite il campo "available" e gestione della concorrenza.

Nello specifico, si può implementare una coda che processi le richieste in modo asincrono. Ogni richiesta sarà connessa a un flusso logico che prima verifica se l'oggetto è disponibile e poi, in caso affermativo, accetta il prestito. Non viene processata la richiesta successiva fino a quando questa non è stata registrata. In caso negativo, la richiesta potrà opzionalmente comunicare all'utente quando l'oggetto richiesto sarà disponibile.